Pre-requisites for Autonomous Systems

Lokesh Mohanty

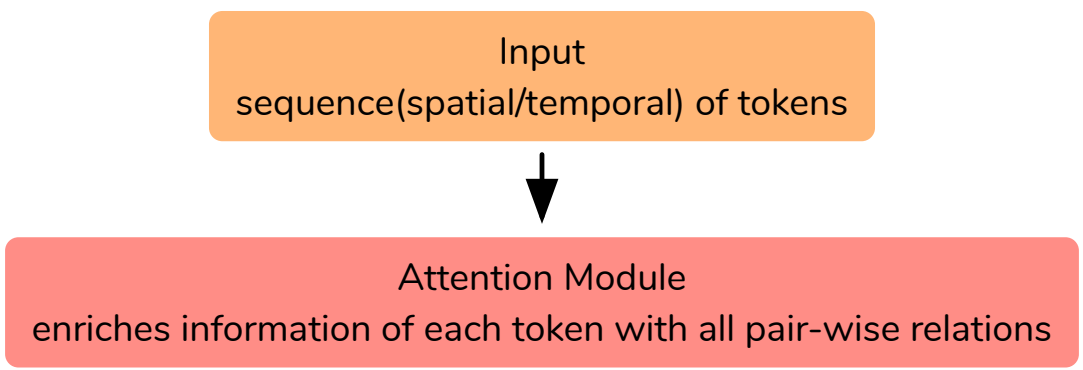
AiREX Lab
Indian Institute of Science

20 Feb, 2026

Contents

Transformer

- This architecture[1] was introduced for Seq-to-Seq tasks
- Excels in learning spatial and temporal relations
- Higly parallelizable leading to fast execution



[1] [Vaswani et. al.] Attention is all you need [3]

Transformer

- Three vectors (query(q_i), key(k_i), value(v_i)) are generated from each input token(x_i)
- Each token queries (q) how much of attention it should give to every other token

Transformer

- Three vectors (query(q_i), key(k_i), value(v_i)) are generated from each input token(x_i)
- Each token queries (q) how much of attention it should give to every other token

$$X \rightarrow \text{input},\ n \rightarrow \text{number of tokens},\ d \rightarrow \text{token dimension}$$

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d,\ W^{\{Q,\ K,\ V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q,\ k_i \rightarrow x_i W^K,\ v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax}\left(\text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}}\right)$$

$$\textbf{Attention } (a_i) \leftarrow pV$$

Transformer

- Three vectors (query(q_i), key(k_i), value(v_i)) are generated from each input token(x_i)
- Each token queries (q) how much of attention it should give to every other token
- In Self-Attention, query and key, value belong to the same sequence
- In Cross-Attention, query belongs to a different sequence

$$X \rightarrow \text{input},\ n \rightarrow \text{number of tokens},\ d \rightarrow \text{token dimension}$$

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d,\ W^{\{Q,\ K,\ V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q,\ k_i \rightarrow x_i W^K,\ v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax}\left(\text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}}\right)$$

$$\textbf{Attention}\ (a_i) \leftarrow pV$$

Transformer

- Three vectors (query(q_i), key(k_i), value(v_i)) are generated from each input token(x_i)
- Each token queries (q) how much of attention it should give to every other token
- In Self-Attention, query and key, value belong to the same sequence
- In Cross-Attention, query belongs to a different sequence

$$X \rightarrow \text{input},\ n \rightarrow \text{number of tokens},\ d \rightarrow \text{token dimension}$$

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d,\ W^{\{Q,\ K,\ V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q,\ k_i \rightarrow x_i W^K,\ v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax}\left(\text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}}\right)$$

$$\textbf{Attention}\ (a_i) \leftarrow pV$$

$$X \in \mathbb{R}^{n \times d},\ Q,\ K,\ V \in \mathbb{R}^{n \times d}$$

$$Q \leftarrow XW^Q,\ K \leftarrow XW^K,\ V \leftarrow XW^V$$

$$\textbf{Attention} \leftarrow \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Transformer

- Three vectors (query(q_i), key(k_i), value(v_i)) are generated from each input token(x_i)
- Each token queries (q) how much of attention it should give to every other token
- In Self-Attention, query and key, value belong to the same sequence
- In Cross-Attention, query belongs to a different sequence
- Attention layers are computationally less expensive than recurrent and convolutional layers

$$X \rightarrow \text{input},\ n \rightarrow \text{number of tokens},\ d \rightarrow \text{token dimension}$$

$$x_i \rightarrow i^{\text{th}} \text{ token} \in \mathbb{R}^d,\ W^{\{Q,\ K,\ V\}} \in \mathbb{R}^{d \times d}$$

$$q_i \rightarrow x_i W^Q,\ k_i \rightarrow x_i W^K,\ v_i \rightarrow x_i W^V$$

$$\text{probs}(p) \leftarrow \text{Softmax}\left(\text{Attention Scores} \leftarrow \frac{q_i K^T}{\sqrt{d}}\right)$$

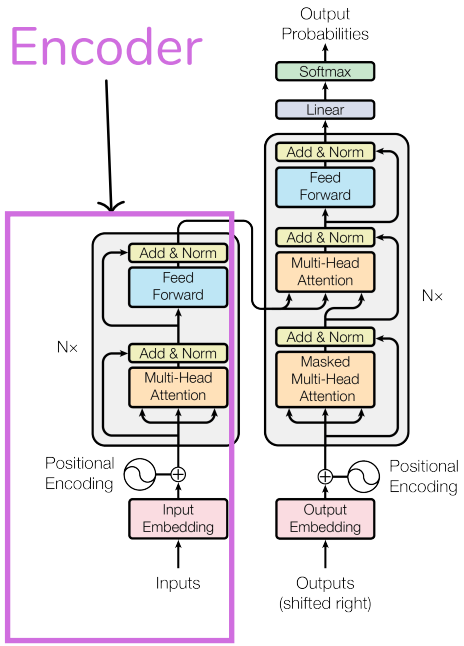
$$\textbf{Attention}\ (a_i) \leftarrow pV$$

$$X \in \mathbb{R}^{n \times d},\ Q,\ K,\ V \in \mathbb{R}^{n \times d}$$

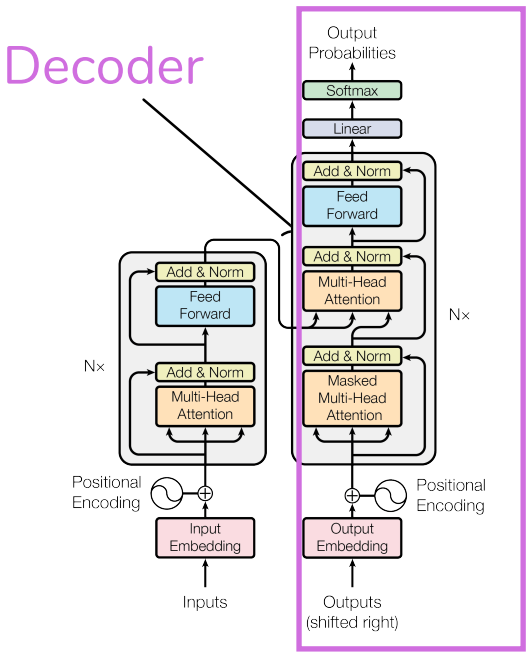
$$Q \leftarrow XW^Q,\ K \leftarrow XW^K,\ V \leftarrow XW^V$$

$$\textbf{Attention} \leftarrow \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Transformer



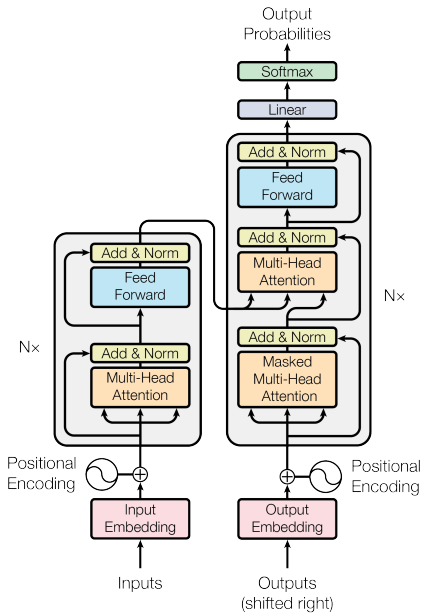
Transformer



Transformer

- In the attention module the attention computed is added

to the input to incorporate contextual information

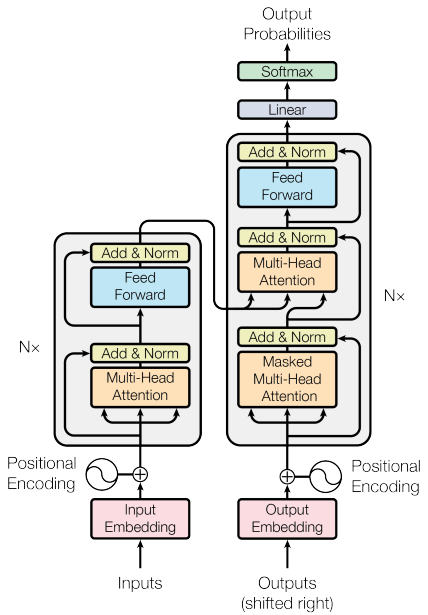


Transformer

- In the attention module the attention computed is added

to the input to incorporate contextual information

- Normalization and Dropout layers are added for regularization

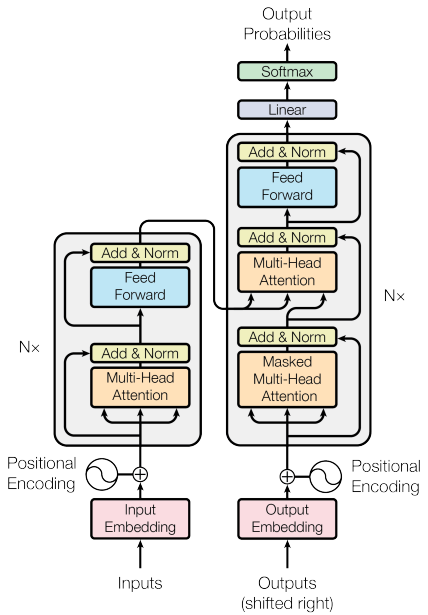


Transformer

- In the attention module the attention computed is added

to the input to incorporate contextual information

- Normalization and Dropout layers are added for regularization
- Feed Forward Networks add non-linearity to the model



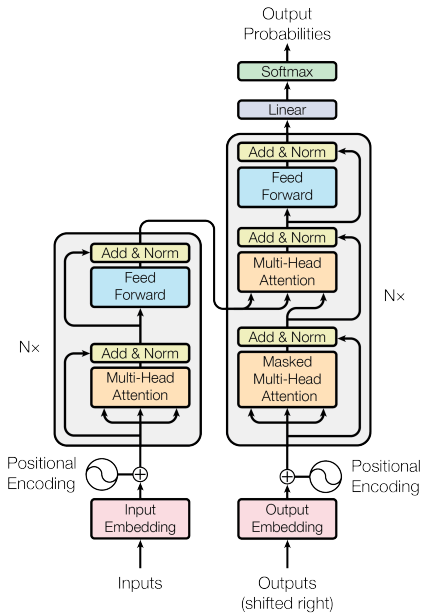
Transformer

- In the attention module the attention computed is added

to the input to incorporate contextual information

- Normalization and Dropout layers are added for regularization
- Feed Forward Networks add non-linearity to the model
- Multiple Attention + FFN layers in sequence form

the backbone of the transformer



Transformer

- Multi-Head allows each head to attend to different feature
- Masking allows hiding information from the model
- It allows the model to process variable sized input
- It also works as a regularizer

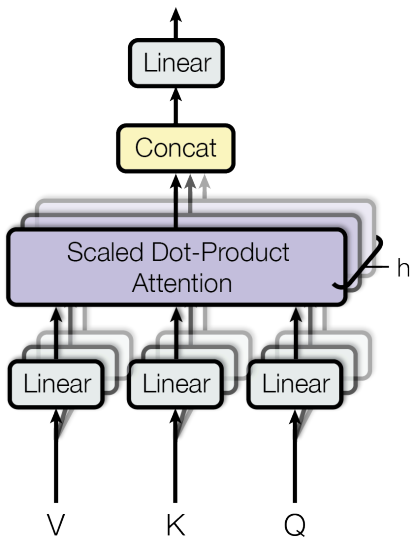


Figure 1: Multi-Head

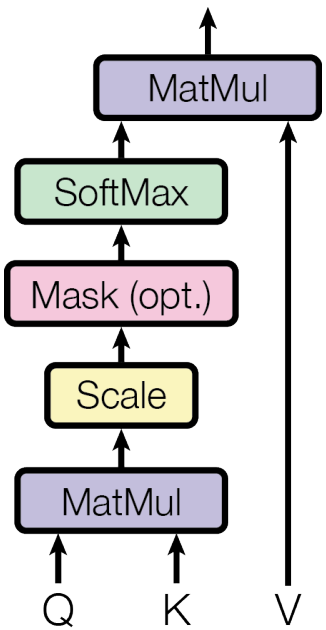


Figure 2: Scaled Dot-Product Attention

Transformer

Absolute Positional Encoding [3] (add to the embedding)

$$\mathrm{PE}_{\mathrm{pos},i} = \begin{cases} \sin\left(\frac{\mathrm{pos}}{1000^{\frac{i}{d_{\mathrm{model}}}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{\mathrm{pos}}{1000^{\frac{i}{d_{\mathrm{model}}}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

$$\mathrm{pos} \leftarrow [0, \mathrm{seq\_length}), i \leftarrow \left[0, \frac{d_{\mathrm{model}}}{2}\right)$$

Rotary Positional Encoding [4] (rotate the embedding)

$$\theta_i = 10000^{-\frac{2i}{d_{\mathrm{model}}}}, i \in \left[1, \frac{d_{\mathrm{model}}}{2}\right]$$

- Effect of positional embedding decreases from the first dimension to the last dimension
- Early dimensions preserve the position information and higher dimensions preseve the token information
- This helps the model to choose which information to attend more

QFormer

Diffusion

The Generative Modeling Problem

Goal: Given samples $\left\{x_0^{(i)}\right\}_{i=1}^{N}$ from an unknown distribution $q(x_0)$, learn a model $p_{\theta(x)}$ such that we can:

- Evaluate $p_{\theta(x)}$ (density estimation)
- Draw new samples $x \sim p_{\theta(x)}$ (generation)

Approach — Latent Variable Models: Introduce latent variables $x_{1:T}$ and define a joint distribution:

$$p_{\theta(x_0)} = \int p_{\theta(x_{0:T})} \, \mathrm{d}x_{1:T}$$

This integral is intractable, so we use variational inference.

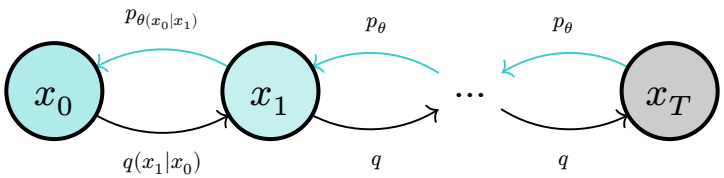
Evidence Lower Bound (ELBO):

$$\log p_{\theta(x_0)} \geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_{\theta(x_{0:T})}}{q(x_{1:T}|x_0)} \right] =: \mathcal{L}_{\mathrm{ELBO}}$$

Maximizing $\mathcal{L}_{\mathrm{ELBO}}$ w.r.t. θ is equivalent to minimizing $D_{\mathrm{KL}(q \parallel p_\theta)}$.

Diffusion

Diffusion — Defining q and p_θ



Forward q: fixed, adds noise | Reverse p_θ: learned, removes noise

Forward (fixed, no params):

$$q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$$

$$q(x_t|x_{t-1}) := \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right)$$

Reverse (learned):

$$p_{\theta(x_{0:T})} := p(x_T) \prod_{t=1}^{T} p_{\theta(x_{t-1}|x_t)}$$

$$p_{\theta(x_{t-1}|x_t)} := \mathcal{N}\left(x_{t-1}; \mu_{\theta(x_t,t)}, \Sigma_{\theta(x_t,t)}\right)$$

where $\beta_t \in (0,1)$ is a noise schedule and $p(x_T) = \mathcal{N}(0, I)$.

Diffusion

Forward Process — Closed-Form Sampling

Define $\alpha_t := 1 - \beta_t$ and $\overline{\alpha}_t := \prod_{s=1}^{t} \alpha_s$. Derive by recursive substitution:

$$
\begin{aligned}
x_1 &= \sqrt{\alpha_1} x_0 + \sqrt{1-\alpha_1}\varepsilon_1 \\
x_2 &= \sqrt{\alpha_2} x_1 + \sqrt{1-\alpha_2}\varepsilon_2 \\
&= \sqrt{\alpha_2\alpha_1} x_0 + \sqrt{1-\alpha_2\alpha_1}\overline{\varepsilon}_2 \quad \text{(merging two Gaussians)} \\
&\vdots \\
x_t &= \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)
\end{aligned}
$$

$$
q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\overline{\alpha}_t} x_0, (1-\overline{\alpha}_t) I\right)
$$

Key properties:

- As $t \to T$, $\overline{\alpha}_t \to 0$, so $q(x_T|x_0) \approx \mathcal{N}(0, I)$ — the data is destroyed
- We can sample any x_t directly from x_0 without iterating (crucial for training)

Posterior is tractable when conditioned on x_0 (by Bayes' rule):

$$
\begin{aligned}
q(x_{t-1}|x_t, x_0) &= \frac{q(x_t|x_{t-1}, x_0) \cdot q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
&\propto \underbrace{\mathcal{N}\big(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I\big)}_{\text{one-step forward}} \cdot \underbrace{\mathcal{N}\Big(x_{t-1}; \sqrt{\overline{\alpha}_{t-1}} x_0, (1-\overline{\alpha}_{t-1}) I\Big)}_{\text{closed-form jump}} \\
&= \mathcal{N}\Big(x_{t-1}; \tilde{\mu}_{t(x_t, x_0)}, \tilde{\beta}_t I\Big) \quad \text{(completing the square)}
\end{aligned}
$$

$$
\tilde{\mu}_t = \frac{\sqrt{\overline{\alpha}_{t-1}}\beta_t}{1-\overline{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t} x_t, \quad \tilde{\beta}_t = \frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}\beta_t
$$

Diffusion

Deriving the Training Objective

Expanding the ELBO for diffusion [5]:

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_q\left[\log p_{\theta(x_0|x_1)}\right]}_{\mathcal{L}_0 \text{ reconstruction}} - \underbrace{D_{\text{KL}(q(x_T|x_0) \parallel p(x_T))}}_{\mathcal{L}_T \approx 0} - \sum_{t=2}^{T} \underbrace{\mathbb{E}_q\left[D_{\text{KL}\left(q(x_{t-1}|x_t,x_0) \parallel p_{\theta(x_{t-1}|x_t)}\right)}\right]}_{\mathcal{L}_{t-1} \text{ denoising matching}}$$

Each $\mathcal{L}_{t-1}$: Match the learned reverse $p_{\theta(x_{t-1}|x_t)}$ to the tractable posterior $q(x_{t-1}|x_t, x_0)$.

Since both are Gaussians, the KL reduces to matching means:

$$\mathcal{L}_{t-1} = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2} \parallel \tilde{\mu}_{t(x_t,x_0)} - \mu_{\theta(x_t,t)} \parallel^2\right] + C$$

Reparametrize μ_θ to predict ε instead of μ directly:

$$\mu_{\theta(x_t,t)} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_{\theta(x_t,t)}\right)$$

Diffusion

Simplified Objective & Training

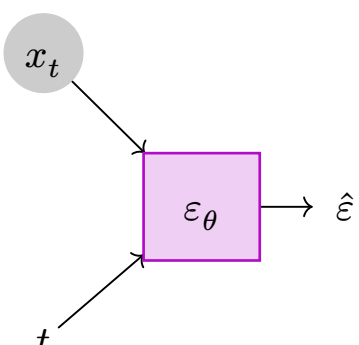
Substituting the reparametrization into $\mathcal{L}_{t-1}$ and dropping constants:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\left[\| \varepsilon - \varepsilon_{\theta(x_t,t)} \|^2\right]$$

where $t \sim \mathcal{U}(\{1,...,T\})$, $\varepsilon \sim \mathcal{N}(0,I)$, $x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon$

Training (per iteration):

1. Sample $x_0 \sim q(x_0)$
2. Sample $t \sim \mathcal{U}(\{1,...,T\}), \varepsilon \sim \mathcal{N}(0,I)$
3. Compute $x_t = \sqrt{\overline{\alpha}_t}x_0 + \sqrt{1-\overline{\alpha}_t}\varepsilon$
4. Gradient step on $\nabla_\theta \| \varepsilon - \varepsilon_{\theta(x_t,t)} \|^2$



Neural network predicts the noise ε from (x_t, t)

Diffusion

DDPM Sampling (Inference)

Starting from $x_T \sim \mathcal{N}(0, I)$, iteratively apply:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta(x_t, t)} \right) + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, I)$$

Algorithm:

1. Sample $x_T \sim \mathcal{N}(0, I)$
2. For $t = T, T-1, ..., 1$:
   - Sample $z \sim \mathcal{N}(0, I)$ if $t > 1$, else $z = 0$
   - $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta(x_t, t)} \right) + \sqrt{\beta_t} z$
3. Return x_0

Problem: Requires $T \approx 1000$ sequential neural network evaluations — very slow.

Diffusion

DDIM — Accelerated Sampling [1]

Key observation: The ELBO only depends on $q(x_t|x_0)$, not the full joint $q(x_{1:T}|x_0)$.

DDIM defines a non-Markovian forward process with the same marginals:

$$q_{\sigma(x_{t-1} \mid x_t, x_0)} = \mathcal{N}\left(\sqrt{\overline{\alpha}_{t-1}}x_0 + \sqrt{1-\overline{\alpha}_{t-1}-\sigma_t^2}\cdot\frac{x_t-\sqrt{\overline{\alpha}_t}x_0}{\sqrt{1-\overline{\alpha}_t}}, \sigma_t^2 I\right)$$

Consequence:

- Uses the same trained ε_θ from DDPM
- Can skip timesteps: use subsequence $\tau_1, ..., \tau_S$ with $S \ll T$
- No retraining needed

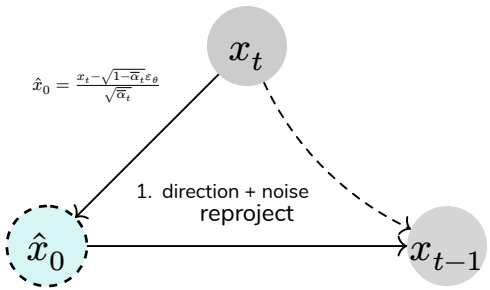
Stochasticity control via $\boldsymbol{\sigma_t}$:

- $\sigma_t = \sqrt{\frac{1-\overline{\alpha}_{t-1}}{1-\overline{\alpha}_t}}\sqrt{\beta_t}$: recovers DDPM
- $\sigma_t = 0$: fully deterministic (ODE)

Diffusion

DDIM — Update Rule

$$x_{t-1} = \underbrace{\sqrt{\overline{\alpha}_{t-1}} \cdot \frac{x_t - \sqrt{1-\overline{\alpha}_t}\varepsilon_{\theta(x_t,t)}}{\sqrt{\overline{\alpha}_t}}}_{\text{predicted } x_0 \text{ scaled to } t-1} + \underbrace{\sqrt{1-\overline{\alpha}_{t-1}-\sigma_t^2} \cdot \varepsilon_{\theta(x_t,t)}}_{\text{direction pointing to } x_t} + \underbrace{\sigma_t \varepsilon_t}_{\text{noise}}$$



When $\sigma_t = 0$: the mapping $x_T \mapsto x_0$ is deterministic, enabling latent interpolation.

Diffusion

DDPM vs DDIM — Summary

	DDPM	DDIM
Forward process	Markovian	Non-Markovian
Marginals $q(x_t \mid x_0)$	Same	Same
Inference steps	1000	50 (20x faster)
Deterministic?	No	Yes ($\sigma_t = 0$)
Latent interpolation	Hard	Easy
Model reuse	—	Uses DDPM's ε_θ

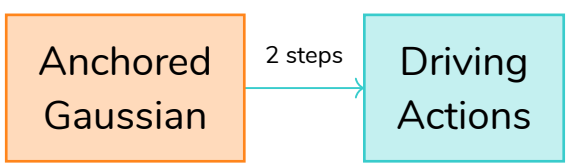
DiffusionDrive [2]

DiffusionDrive — Motivation

Problem: Vanilla diffusion policies are too slow for real-time driving.

- DDPM/DDIM require 20+ denoising steps
- Driving requires real-time ($>$ 30 FPS) multi-modal planning
- Traffic scenes are open-world — the action space is highly multi-modal

Key Idea: Don't start from pure noise $\mathcal{N}(0, I)$. Instead, start from an anchored Gaussian close to the target distribution, drastically reducing denoising from 20 steps to just 2.



vs. vanilla: $\mathcal{N}(0, I) \overset{20 \text{ steps}}{\rightarrow}$ actions

Two innovations:

1. Truncated diffusion with multi-modal anchors
2. Efficient cascade diffusion decoder

DiffusionDrive [2]

Truncated Diffusion Policy with Multi-Modal Anchors

Step 1 — Build anchors:

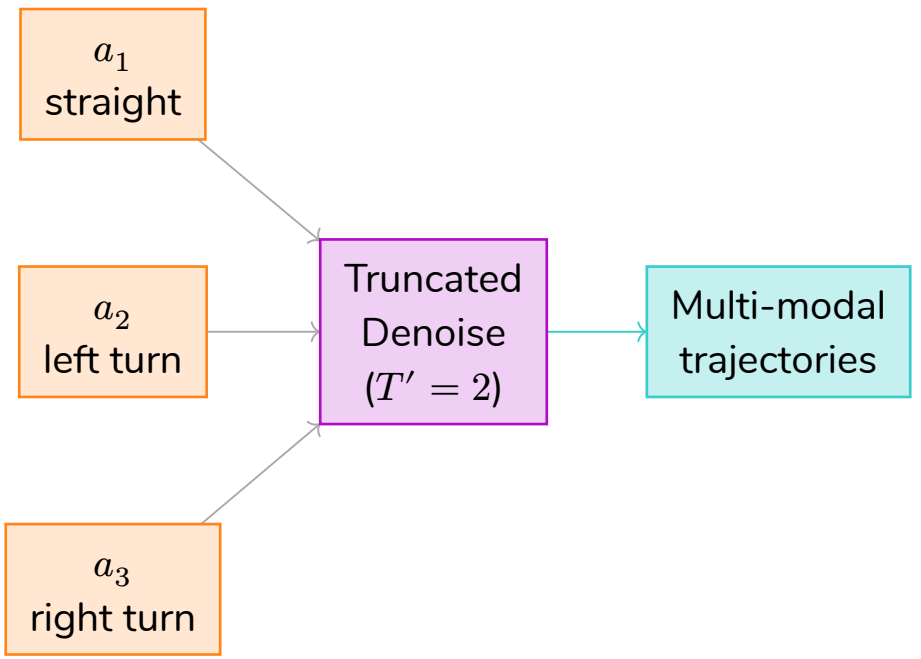
- Cluster expert driving trajectories into K modes (e.g., straight, left turn, right turn, lane change)
- Each cluster centroid a_k becomes an anchor

Step 2 — Truncated forward:

- Instead of noising for T steps to $\mathcal{N}(0, I)$, only noise for T' steps ($T' \ll T$)
- Start from $q(x_{T'} \mid a_k)$ — a noisy version of the anchor, not pure noise

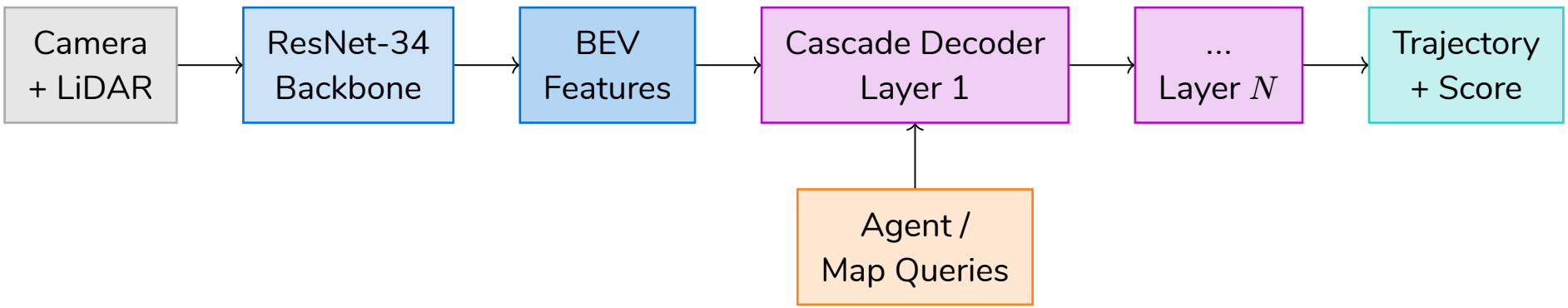
Step 3 — Truncated reverse:

- Denoise from $x_{T'} \sim \mathcal{N}\left(\sqrt{\overline{\alpha}_{T'}} a_k, (1 - \overline{\alpha}_{T'}) I\right)$
- Only $T' = 2$ denoising steps needed!



DiffusionDrive [2]

Cascade Diffusion Decoder



Each cascade layer performs:

- Deformable cross-attention between trajectory waypoints and BEV features
- Cross-attention with agent and map queries for scene context
- MLP head outputs incremental trajectory offset $\Delta\tau$ and confidence score

DiffusionDrive [2]

DiffusionDrive — Results

Metric	Value
NAVSIM PDMS	88.1 (SOTA with ResNet-34)
Denoising steps	2 (vs. 20 for vanilla diffusion)
Inference speed	45 FPS (NVIDIA 4090)
Speedup	10× vs. vanilla diffusion policy

Key takeaways:

- Multi-modal anchors from expert clustering provide strong priors
- Truncated diffusion bridges “near-mode” gap in just 2 steps
- Cascade decoder enables rich scene-conditioned refinement
- Real-time capable — practical for deployment

Bibliography

[1] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *International Conference on Learning Representations*, 2021.

[2] B. Liang, P. Jia, J. Chen, Y. Jiang, and others, “DiffusionDrive: Truncated Diffusion Model for End-to-End Autonomous Driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[3] A. Vaswani *et al.*, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 6000–6010.

[4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” [Online]. Available: https://arxiv.org/abs/2104.09864

[5] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems*, 2020, pp. 6840–6851.